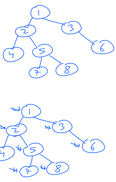
Trees

Node {
int data;
vector<Node*> children;
}

Binary Trees

Node {
int data;
Node* left;
Node* right;
}

Preorder



1) Root
2) Preorder of LT
3) Preorder of RT

TC: O(n).
AS: O(H)

[1, 2, 4, 5, 7, 8, 3, 6].

Postorder

1) Postorder of LT
2) Postorder of RT
3) Root

TC: O(n)
AS: O(H)

[4, 7, 8, 5, 2, 6, 3, 1].

[4, 5, 2, 8, 6, 9, 7, 3, 1]

[4 5 2 8 6 9 7 3 1]

[4 5 2 8 6 7 9 3 1]

Inorder

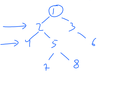
1) Inorder of LT
2) root
3) Inorder of RT

[4, 2, 7, 5, 8, 1, 3, 6].

TC: O(n).
AS: O(H).

[4 2 5 1 8 6 3 7 9]

Level order



g: 1 2 3 4 5 6 7 8

arr: [1, 2, 3, 4, 5, 6, 7, 8]

TC: O(n).
AS:

H = log n

AS: $O(2^{\log n - 1})$

$\frac{2^{\log n}}{2} = \frac{n}{2}$

$= O(\frac{n}{2}) = O(n)$.

log n ~ 3.
H: 3

H = 7 = n

AS: O(H)

Level-by-level Levelorder

[[1],
[2, 3],
[4, 5, 6],
[7, 8]]

g: 1 2 3 4 5 6 7 8

q: size = 2 3 2

[2, 3]
[4, 5, 6]
[- - -]

```
while (!q.empty()) {
    int sz = q.size();
    vector<int> currentLevel;

    for (int i = 0; i < sz; i++) {
        Node curr = q.front();
        q.pop();

        currentLevel.push_back(curr->data);

        if (curr->left != NULL) {
            q.push(curr->left);
        }
        if (curr->right != NULL) {
            q.push(curr->right);
        }
    }

    result.push_back(currentLevel);
}
```

g: 1 2 3 4 5 6 7 8

sz = 2 2 2 2

curr level: [1] [2, 3] [4, 5, 6] [7, 8]

res: [[1]
[2,3]
[4,5,6]
[7,8]]

curr: 1 2 3 4 5 6 7 8

Height

Ht(root) = 1 + max(Ht(root->left), Ht(root->right))